Práctica 0: Primer contacto con la Raspberry PI

Belmont Muñoz Samuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 317318289

Domínguez Palacios Jesús Alejandro
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 421066205

Dorantes Mejía Emmanuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 313004766

García Olvera Emiliano
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 314256009

Hernández Alonso Antonio Adonahí
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 316339825

Jiménez Reyes Alan Yael
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 320026726

Práctica No.: 0 | **Grupo:** 6 | **Fecha:** 31/08/2026

***Resumen*—Este reporte presenta una práctica introductoria sobre sistemas embebidos utilizando la Raspberry Pi 4 como plataforma de desarrollo. Se describe el proceso de instalación y configuración del sistema operativo Raspberry Pi OS en modo headless, habilitando acceso remoto mediante SSH. El documento aborda las diferencias entre sistemas embebidos y computadoras de propósito general, así como la ubicación de la Raspberry Pi 4 dentro del espectro de plataformas del curso. Se incluye la metodología para realizar la configuración inicial, establecer conexión remota y ejecutar comandos básicos de verificación del sistema.**

***Index Terms*—Sistemas embebidos, Raspberry Pi 4, SSH, Raspberry Pi OS, configuración headless**

I. Objetivo

Los objetivos de esta práctica son:

- Distinguir un sistema embebido de una computadora de propósito general, con ejemplos reales.
- Ubicar la Raspberry Pi 4 y el STM32 dentro del panorama de plataformas del curso.
- Instalar Raspberry Pi OS en una microSD con Raspberry Pi Imager, configurando SSH y red sin monitor ni teclado (headless).
- Realizar el primer arranque de la Raspberry Pi y conectarse a ella por SSH desde su propia laptop.
- Ejecutar comandos básicos de verificación de hardware y sistema operativo.

II. Introducción

Los sistemas embebidos son dispositivos computacionales diseñados para realizar funciones específicas dentro de un sistema mayor, a diferencia de las computadoras de propósito general que pueden ejecutar una amplia variedad de aplicaciones. Ejemplos de sistemas embebidos incluyen controladores de electrodomésticos, sistemas de control automotriz, dispositivos IoT, entre otros.

La Raspberry Pi 4 es una computadora de placa única (SBC, Single Board Computer) que se encuentra en el punto intermedio entre microcontroladores simples y computadoras completas. Cuenta con un procesador ARM de múltiples núcleos, memoria RAM, puertos USB, Ethernet, HDMI y GPIO (General Purpose Input/Output), lo que la hace ideal para proyectos de sistemas embebidos con requisitos de procesamiento moderados a altos.

En el contexto del curso, la Raspberry Pi 4 representa una plataforma de alto nivel donde se puede ejecutar un sistema operativo completo (Linux), mientras que el STM32 es un microcontrolador de menor nivel que opera sin sistema operativo (bare-metal) o con sistemas operativos en tiempo real (RTOS).

Para trabajar con la Raspberry Pi sin necesidad de conectar periféricos como monitor o teclado, se utiliza una configuración conocida como "headless". Esta configuración implica habilitar el servicio SSH (Secure Shell) desde la instalación inicial del sistema operativo, permitiendo el acceso remoto a través de la red. Raspberry Pi OS, basado en Debian Linux, proporciona todas las herramientas necesarias para administrar y programar la placa de forma remota.

III. Desarrollo

III-A. Material y Equipo

Para la realización de esta práctica se utilizó el siguiente material y equipo:

- Computadora personal (Host)
- Raspberry Pi 4 Model B
- Tarjeta de memoria microSD (mínimo 16 GB recomendados)
- Lector de tarjetas microSD
- Software Raspberry Pi Imager

III-B. Procedimiento

III-B1. Instalación del Sistema Operativo con Raspberry Pi Imager: El proceso de configuración inició con la preparación del medio de almacenamiento utilizando la herramienta oficial Raspberry Pi Imager. En la interfaz principal del software, se seleccionó el dispositivo correspondiente a flashear (Fig. 1). A continuación, se desplegó el catálogo de sistemas operativos disponibles y se seleccionó la imagen "Raspberry Pi OS (64-bit)", basada en Debian, recomendada para aprovechar la arquitectura del procesador de la Raspberry Pi 4 (Fig. 2).

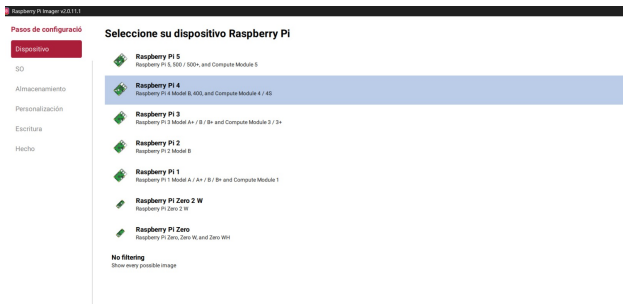


Figura 1. Pantalla inicial de Raspberry Pi Imager y selección del dispositivo.

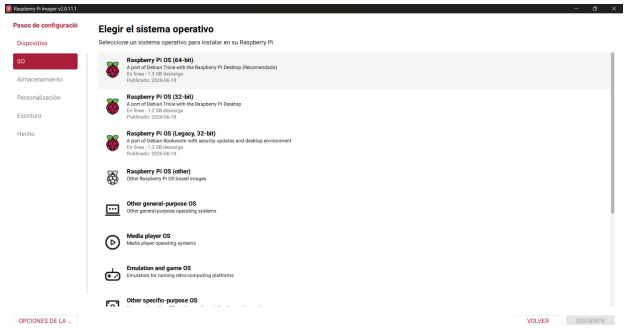


Figura 2. Selección de la imagen del sistema operativo (Raspberry Pi OS 64-bit).

Antes de proceder con la escritura en la memoria microSD, fue necesario aplicar las configuraciones avanzadas para habilitar el modo *headless*. Dentro del menú de personalización del sistema operativo, se habilitó el servicio SSH, se definió el nombre de host (hostname) y se establecieron las credenciales de acceso (usuario y contraseña). Asimismo, se configuró la conexión a la red local. Por último, se ejecutó el proceso de escritura y verificación en la tarjeta.

III-B2. Conexión Remota vía SSH (Configuración Headless): Una vez flasheada la tarjeta microSD, se insertó en la ranura de la Raspberry Pi 4 y se energizó la placa. Tras otorgar un margen de tiempo para el primer arranque y la asignación de red, se procedió a establecer la conexión remota desde la computadora host.

Utilizando el intérprete de comandos Windows PowerShell, se ejecutó el comando de conexión mediante el protocolo SSH, especificando el usuario y la dirección IP asignada localmente (`ssh konan@192.168.1.97`). Durante este primer intento de conexión, el sistema solicitó la confirmación para añadir la huella digital criptográfica (ED25519 key fingerprint) a la lista de hosts conocidos del equipo cliente; tras confirmar e ingresar la contraseña, el acceso fue exitoso (Fig. 3).

Para validar la correcta ejecución de los servicios de red en el sistema embebido, se comprobó el estado del demonio de SSH mediante el comando `sudo systemctl status ssh`. La salida de la terminal confirmó que el servicio *OpenBSD Secure Shell server* se encontraba activo y en ejecución (Fig. 4).

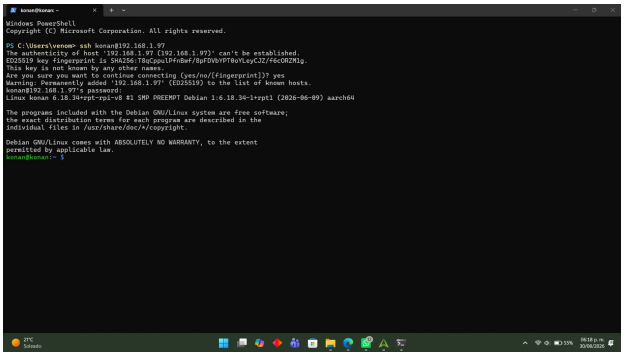


Figura 3. Conexión inicial por SSH desde Windows PowerShell y aceptación del certificado.

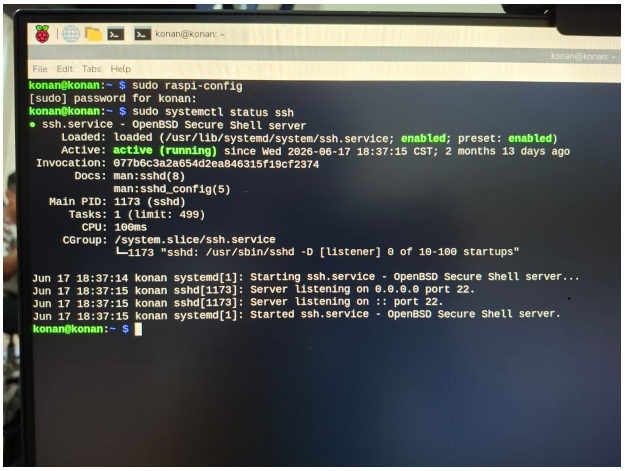


Figura 4. Verificación del estado del servicio SSH dentro de la Raspberry Pi.

III-B3. Verificación del Sistema y Hardware: Con la sesión SSH establecida correctamente (Fig. 6), se procedió a ejecutar una serie de comandos de diagnóstico indicados en la guía de la práctica (Fig. 5) para inspeccionar las características del hardware y del sistema operativo recién instalado.

En primer lugar, se ejecutaron los comandos `uname -a` y `cat /proc/cpuinfo` (Fig. 7). El primer comando permitió confirmar que el sistema ejecuta un kernel de Linux en su versión 6.1 bajo la arquitectura `aarch64` (64 bits). Por su parte, la lectura del archivo `cpuinfo` arrojó detalles específicos del procesador, revelando la presencia de 4 núcleos (numerados del 0 al 3) e identificando la placa como una *Raspberry Pi 4 Model B Rev 1.5*.

Para inspeccionar la memoria del sistema, se consultó el archivo `/proc/meminfo` (Fig. 8), el cual despliega un desglose exhaustivo de la memoria RAM, indicando un total aproximado de 927 MB (correspondiente al modelo de 1 GB de RAM), así como la distribución de memoria en búferes y caché.

Posteriormente, se utilizó el comando `lscpu` (Fig. 9) para obtener un resumen más legible de la arquitectura. Este comando confirmó el uso de un procesador ARM Cortex-A72 con un orden de bytes *Little Endian* y una frecuencia máxima

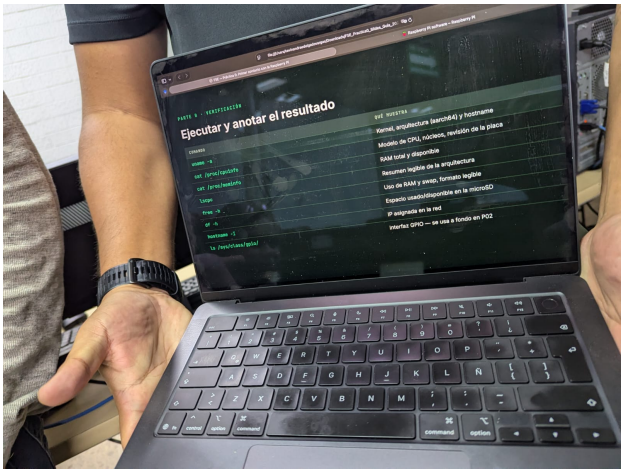


Figura 5. Lista de comandos de verificación de sistema propuestos para la práctica.

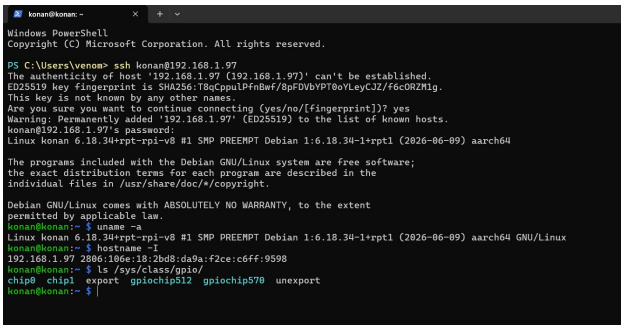


Figura 6. Conexión remota establecida y visualización del prompt operativo.

de operación de 1800 MHz (1.8 GHz).

Como último paso, se ejecutó un último bloque de comandos de monitoreo general (Fig. 10). Se utilizó `free -h` para observar de forma resumida y legible el consumo de memoria RAM y Swap. El comando `df -h` se empleó para verificar las particiones y el espacio de almacenamiento en la tarjeta microSD, mostrando una partición principal de aproximadamente 59 GB. Mediante `hostname -I` se verificó nuevamente la dirección IP asignada en la red local (192.168.1.97). Para concluir la verificación orientada a sistemas embebidos, se listó el contenido del directorio `/sys/class/gpio/` mediante el comando `ls`, confirmando la disponibilidad de las interfaces lógicas para el control de los pines GPIO, fundamentales para prácticas futuras.

IV. Resultados y Análisis

A partir de la ejecución de los comandos de diagnóstico y la inspección del sistema durante la sesión SSH, se consolidaron los parámetros técnicos más representativos de la plataforma de desarrollo. Los datos obtenidos del hardware y del entorno de ejecución se resumen en el Cuadro I.

IV-A. Análisis de la Arquitectura y Procesador

La información devuelta por los utilitarios `uname`, `cpuinfo` y `lscpu` confirma la operatividad en modo de

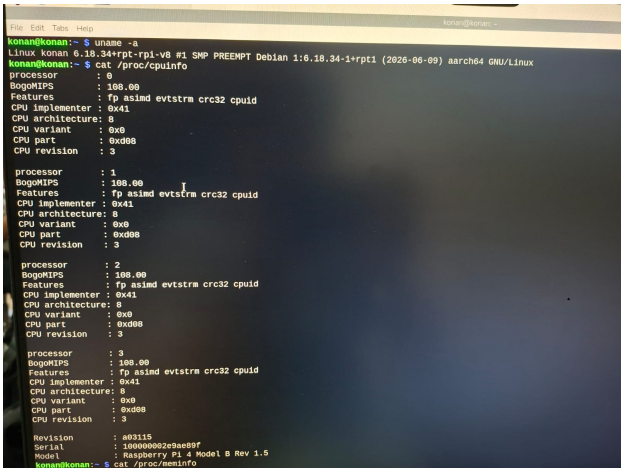


Figura 7. Salida de los comandos `uname -a` y `cat /proc/cpuinfo`.

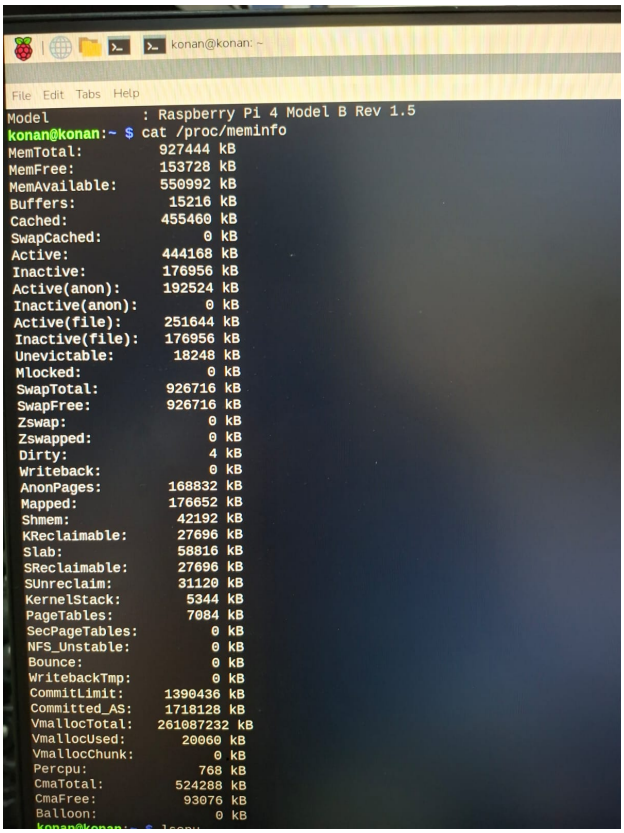


Figura 8. Despliegue del estado de la memoria RAM mediante `cat /proc/meminfo`.

64 bits (`aarch64`). La microarquitectura Cortex-A72 opera con un rango de frecuencia dinámico escalable entre 600 MHz y 1.8 GHz gracias al gobernador de frecuencia del kernel Linux, lo cual es fundamental en sistemas embebidos modernos para optimizar el consumo energético y la disipación térmica cuando la carga de trabajo es baja. La presencia de 4 núcleos independientes permite la concurrencia real y la ejecución de tareas críticas en paralelo mediante un sistema

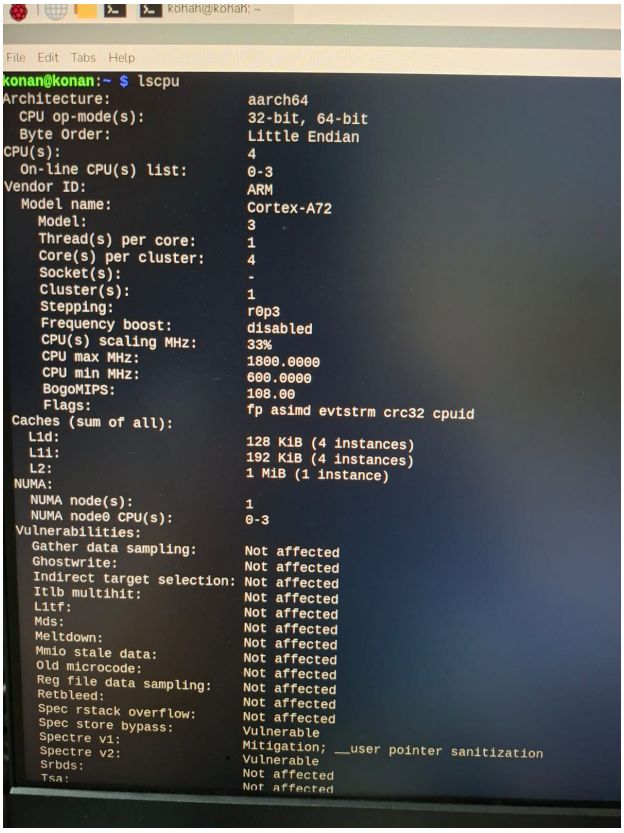


Figura 9. Resumen legible de la arquitectura del procesador obtenido con `lscpu`.

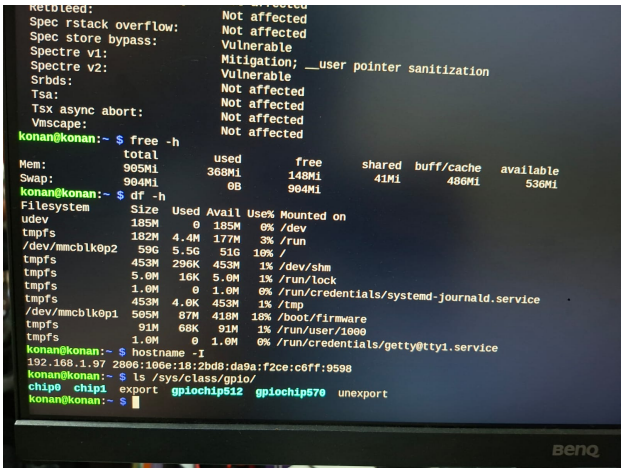


Figura 10. Verificación de memoria, almacenamiento, IP e interfaz GPIO.

operativo multitarea.

IV-B. Gestión de Memoria y Almacenamiento

El análisis del comando `cat /proc/meminfo` y su contraparte resumida `free -h` evidencia que, tras el arranque inicial sin entorno gráfico interactivo de escritorio (modo puramente de terminal sobre SSH), el sistema operativo base consume aproximadamente 368 MiB. Esto deja disponibles más de 530 MiB para aplicaciones de usuario y procesos en segundo plano.

Cuadro I
PARÁMETROS TÉCNICOS Y MÉTRICAS DEL SISTEMA OBTENIDOS

Parámetro / Métrica	**Valor / Descripción Obtenida**
Placa y Revisión	Raspberry Pi 4 Model B Rev 1.5
Arquitectura del CPU	ARM Cortex-A72 (`aarch64` / ARMv8)
Núcleos / Hilos	4 núcleos físicos / 4 hilos (1 por núcleo)
Frecuencia de Operación	600 MHz (mín.) – 1800 MHz (máx.)
Memoria RAM Total	927 MiB (≈ 1 GB)
Memoria RAM en Uso	368 MiB (≈ 39.7 % del total)
Almacenamiento (Raíz `/`)	59 GB totales (5.5 GB en uso, 51 GB disp.)
Kernel de Linux	6.18.34+rpt-rpi-v8 (Debian Trixie)
Dirección IP (Host)	192.168.1.97

Por otro lado, el comando `df -h` reflejó que el sistema de archivos principal montado en `/dev/mmcblk0p2` aprovecha adecuadamente la capacidad completa de la tarjeta microSD (aproximadamente 59 GB libres para almacenamiento de binarios, logs y dependencias).

IV-C. Interoperabilidad y Subsistema GPIO

El listado del directorio virtual `/sys/class/gpio/` valida la existencia de la interfaz *sysfs* del kernel para la exportación y control de líneas de entrada/salida de propósito general. Aunque en las versiones recientes de Linux se promueve la migración hacia interfaces basadas en `libgpiod` y *character devices* (`/dev/gpiochipX`), la presencia de los controladores `gpiochip512` y `gpiochip570` confirma que el sistema embebido está correctamente inicializado para interactuar a nivel de hardware con sensores, actuadores y protocolos de comunicación física en prácticas posteriores.

IV-D. Análisis de Sistemas Operativos Disponibles para Raspberry Pi

IV-D1. ¿Cuántos sistemas operativos son?: Raspberry Pi Imager ofrece una gran cantidad de sistemas operativos, agrupados en categorías. Entre las principales se encuentran:

- **Raspberry Pi OS** (versiones de 32 y 64 bits) — el sistema oficial y recomendado.
- **Sistemas operativos de propósito general**: Ubuntu, Manjaro Linux, RISC OS.
- **Centros multimedia**: LibreELEC, OSMC.
- **Emulación de videojuegos**: RetroPie, Recalbox, Batocera.
- **Categorías especializadas**: Digital signage and kiosks, Home automation y 3D printing, que a su vez contienen submenús con más sistemas operativos específicos (RaspAP, eBlockerOS, Falcon Player, FreedomBox, MoodleBox, Clea OS, SatNOGS, Kali Linux, entre otros).

Por lo tanto, no se trata de un número fijo, sino de un catálogo extenso y en constante crecimiento de imágenes, tanto oficiales como de terceros.

IV-D2. ¿Para qué están hechos?: Cada sistema operativo está diseñado para un propósito particular. Por ejemplo, Raspberry Pi OS es de uso general y mantiene compatibilidad total con el hardware de la tarjeta; Ubuntu y Manjaro ofrecen un entorno Linux completo similar al de una computadora convencional; LibreELEC y OSMC están orientados a centros multimedia; y las categorías especializadas (domótica, señalización digital, impresión 3D, redes, seguridad informática, etc.) contienen sistemas optimizados exclusivamente para esas tareas.

IV-D3. ¿Por qué no podemos usar cualquier SO para lo que haremos?: Porque cada imagen está construida y configurada en función de su propósito específico, priorizando el software, los servicios y las optimizaciones necesarias para esa tarea, y no necesariamente el acceso completo, estable y documentado a la interfaz GPIO, que es indispensable para el trabajo con sensores y proyectos embebidos en este curso. Usar un sistema orientado a otra finalidad (por ejemplo, un media player o un emulador de videojuegos) podría implicar que el GPIO no esté disponible, no cuente con las librerías adecuadas o simplemente no sea el enfoque idóneo para el desarrollo que se realizará.

IV-D4. Si no tuviéramos Raspberry Pi OS, ¿qué otro SO nos permite usar los GPIO?: Ubuntu, al tratarse de un sistema operativo de propósito general basado en Linux con soporte oficial para Raspberry Pi, permite el acceso a la interfaz GPIO mediante las librerías correspondientes. De manera similar, Manjaro Linux, por su naturaleza de distribución Linux completa, también constituye una alternativa viable para el manejo de GPIO en caso de no contar con Raspberry Pi OS.

V. Conclusiones

Belmont Muñoz Samuel. A través del desarrollo de esta práctica introductoria, se cumplieron satisfactoriamente los objetivos planteados. Se logró comprender de forma práctica la posición de la Raspberry Pi 4 dentro del espectro de los sistemas embebidos, reconociendo sus capacidades como computadora de placa única (SBC) de alto desempeño frente a plataformas basadas en microcontroladores de menor nivel.

Asimismo, se validó la eficacia del despliegue en modo *headless*. La configuración previa de parámetros de red y credenciales de acceso mediante Raspberry Pi Imager facilitó el enlace remoto vía SSH desde el equipo host sin requerir el uso de periféricos dedicados. También, la ejecución de los comandos de diagnóstico en terminal permitió corroborar la arquitectura ARM Cortex-A72 del procesador, la gestión eficiente de la memoria en un entorno Linux de 64 bits y la disponibilidad de la interfaz del subsistema GPIO, estableciendo las bases operativas necesarias para el desarrollo de las prácticas posteriores del curso.

Domínguez Palacios Jesús Alejandro. Con el desarrollo de esta práctica quedó clara la distinción entre un sistema embebido y una computadora de propósito general, así como el lugar intermedio que ocupa la Raspberry Pi 4: si bien ejecuta una distribución completa de Linux, en este curso se aprovechará como plataforma dedicada a tareas concretas, lo que la acerca al comportamiento de un sistema embebido.

La instalación de Raspberry Pi OS se realizó con Raspberry Pi Imager, aprovechando las opciones de configuración previa a la escritura de la microSD para habilitar SSH, registrar las credenciales de la red Wi-Fi y definir el hostname del equipo. Gracias a ello no fue necesario conectar teclado ni monitor: al primer arranque la tarjeta se integró a la red y se pudo abrir la sesión remota por SSH sin contratiempos.

Ya dentro del sistema, con una serie de comandos básicos de consola se comprobó el estado del hardware y del software de la tarjeta: arquitectura, características del procesador, memoria RAM disponible, espacio de almacenamiento, dirección IP asignada y acceso a los pines GPIO. Con esta verificación se confirma que la Raspberry Pi 4 quedó correctamente preparada para las prácticas siguientes de la asignatura.

Dorantes Mejía Emmanuel. Esta práctica me permitió conocer de forma básica el funcionamiento de la Raspberry Pi 4 y comprender su utilidad en sistemas embebidos. Aprendí a instalar y configurar Raspberry Pi OS, establecer la conexión Wi-Fi y utilizar SSH para acceder de manera remota sin necesidad de monitor o teclado.

También utilicé comandos de Linux para revisar características como el procesador, la memoria, el almacenamiento y los pines GPIO. En general, la práctica sirvió para dejar la Raspberry Pi configurada y lista para utilizarla en futuros proyectos con sensores, actuadores y otros dispositivos electrónicos.

García Olvera Emiliano. En esta práctica se identificaron las diferencias entre un sistema embebido y una computadora de propósito general, ubicando a la Raspberry Pi 4 como un caso particular que, aunque corre Linux completo, se usará de forma dedicada para tareas específicas del curso.

Se logró instalar Raspberry Pi OS mediante Raspberry Pi Imager, configurando previamente SSH, Wi-Fi y hostname para permitir el arranque en modo headless. Tras el primer encendido, se estableció conexión remota por SSH exitosamente.

Finalmente, mediante comandos básicos se verificó el hardware y software de la tarjeta —arquitectura, CPU, memoria, almacenamiento, IP asignada y disponibilidad de GPIO—, confirmando que el equipo quedó listo para las prácticas posteriores del curso.

Hernández Alonso Antonio Adonahí. Esta práctica permitió adquirir experiencia directa en la configuración y puesta en marcha de sistemas embebidos basados en Linux. La metodología de instalación headless demostró ser una estrategia eficaz para la administración remota de dispositivos sin interfaz gráfica, práctica común en entornos de IoT y sistemas embebidos industriales.

El proceso de verificación del hardware mediante comandos de terminal no solo confirmó las especificaciones técnicas de la Raspberry Pi 4, sino que también familiarizó al usuario con herramientas esenciales del entorno Linux para el diagnóstico y monitoreo de sistemas. La comprobación del subsistema GPIO resulta particularmente relevante, pues establece la

disponibilidad de interfaces físicas para interactuar con componentes electrónicos externos en prácticas futuras.

En conclusión, se logró establecer una plataforma de desarrollo completamente funcional y lista para implementar proyectos de sistemas embebidos, cumpliendo satisfactoriamente con los objetivos establecidos al inicio de la práctica.

Jiménez Reyes Alan Yael. Con esta práctica me quedó mucho más clara la diferencia entre una computadora de uso general y un sistema embebido. Pude ver cómo la Raspberry Pi 4 es un excelente punto intermedio: aunque corre un sistema Linux completo de 64 bits, la utilizaremos dedicada a tareas específicas, a diferencia de opciones más básicas como el microcontrolador STM32.

Configurar la tarjeta en modo headless con Raspberry Pi Imager me pareció bastante práctico, ya que nos ahorró tener que conseguir un monitor y teclado extra. Al dejar listos los datos de la red y el acceso SSH desde el principio, la conexión remota fue muy rápida y directa.

Ya en la terminal, ejecutar los comandos me sirvió para revisar los recursos reales de la placa (como el procesador ARM y el uso de RAM) y confirmar que los pines GPIO están detectados. En resumen, el equipo quedó perfectamente preparado y funcional para los proyectos que haremos más adelante.

REFERENCIAS

[1] IEEE. (2024). "Embedded Systems", *IEEE Technology Navigator*. Disponible en: https://technav.ieee.org/topic/embedded-systems/

[2] Raspberry Pi Foundation. (2024). "Getting started with your Raspberry Pi", *Raspberry Pi Documentation*. Disponible en: https://www.raspberrypi.com/documentation/computers/getting-started.html

[3] Arm Limited. (2024). "Cortex-A72 Processor", *Arm Developer Documentation*. Disponible en: https://developer.arm.com/Processors/Cortex-A72

[4] The kernel development community. (2024). "The /proc Filesystem", *Linux Kernel Documentation*. Disponible en: https://www.kernel.org/doc/html/latest/filesystems/proc.html

[5] The kernel development community. (2024). "GPIO in the Linux Kernel", *Linux Kernel Documentation*. Disponible en: https://www.kernel.org/doc/html/latest/admin-guide/gpio/index.html